



Program Analysis via Multiple Context Free Language Reachability

GIOVANNA KOBUS CONRADO

Hong Kong University of Science and Technology

ADAM HUSTED KJELSTRØM

ANDREAS PAVLOGIANNIS

JACO VAN DE POL

Aarhus University

Language Reachability for Static Analysis

Problem

Can the value of variable **x** flow into variable **y**?

This type of problem is found in:

- Data-flow analysis
- Taint analysis
- Data-dependence analysis
- Alias analysis
- ...

One solution

Can the value of variable **a** flow into variable **c**?

```
int  a ← 2
```

```
int  b ← a
```

```
int  c ← b
```

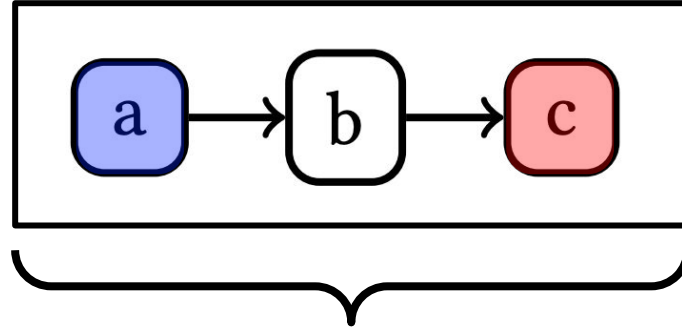
One solution

Can the value of variable **a** flow into variable **c**?

```
int a ← 2
```

```
int b ← a
```

```
int c ← b
```



Check if **a** reaches **c**

What if there are function calls?

Which variables affect the value of **b**?

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```

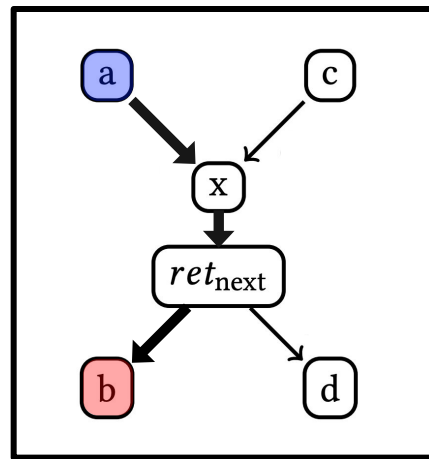
What if there are function calls?

Which variables affect the value of **b**?

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```



a reaches **b**...

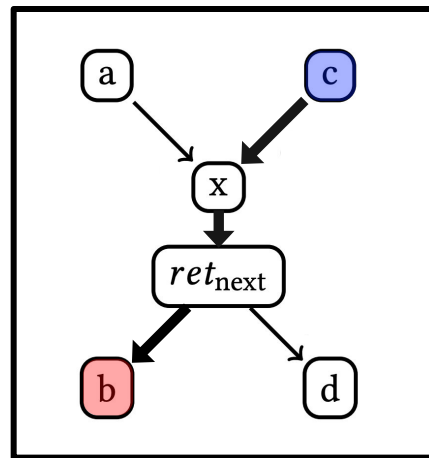
What if there are function calls?

Which variables affect the value of **b**?

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```



a reaches **b**... but so does **c**

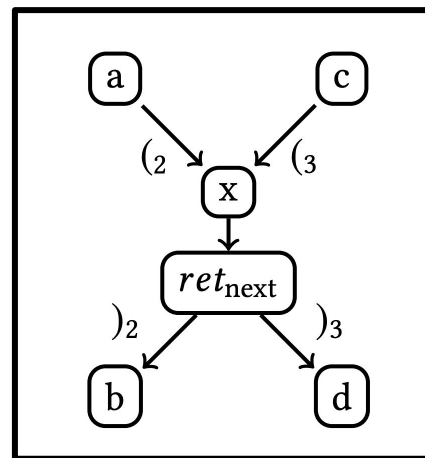
Context-sensitivity

We use parentheses to model calling contexts

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```



the path must be well-nested

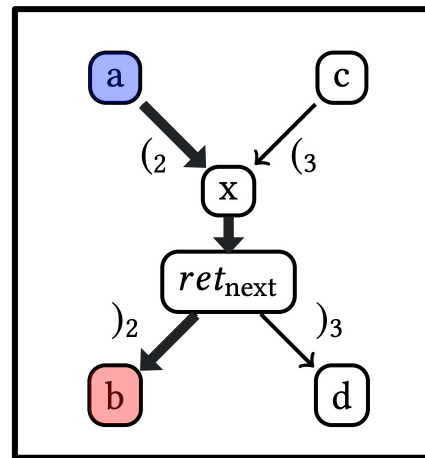
Context-sensitivity

We use parentheses to model calling contexts

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```



a to **b** is context-sensitive $\rightarrow$ word $(_2)_2$

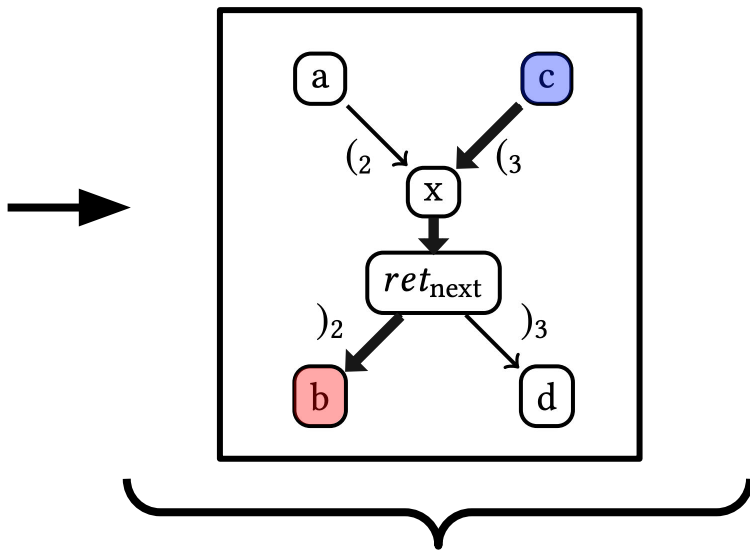
Context-sensitivity

We use parentheses to model calling contexts

```
fun next(x)
  return x+1

fun next2()
  int a ← 2
  int b ← next(a)

fun next3()
  int c ← 3
  int d ← next(c)
```

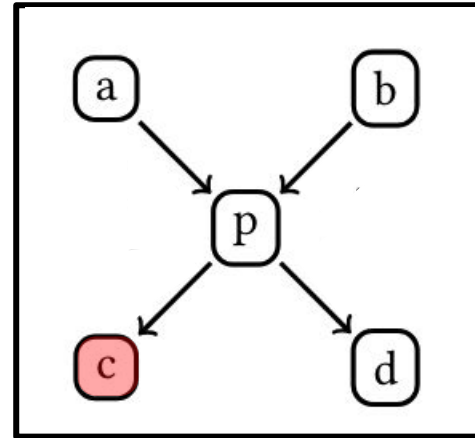


c to **b** is not context-sensitive $\rightarrow$ word $(3)_2$

What if there are composite data types?

Which variables affect the value of **c**?

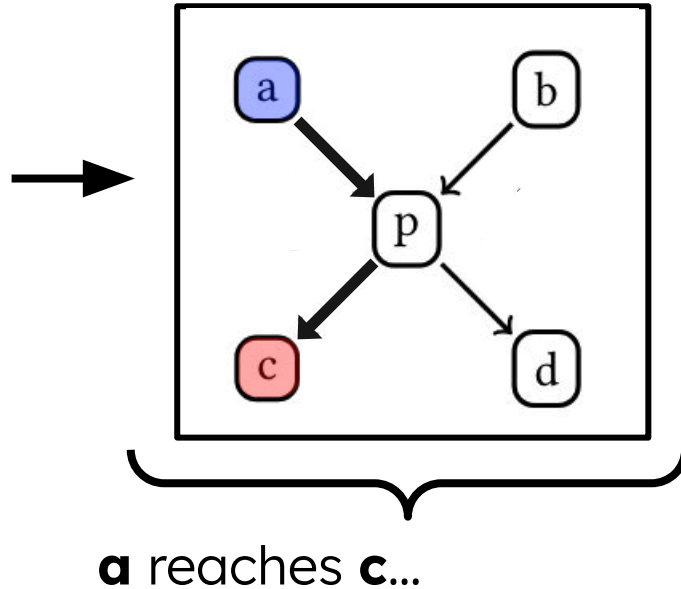
```
int a ← 2  
int b ← 3  
point p  
p.x ← a  
p.y ← b  
int c ← p.x  
int d ← p.y
```



What if there are composite data types?

Which variables affect the value of **c**?

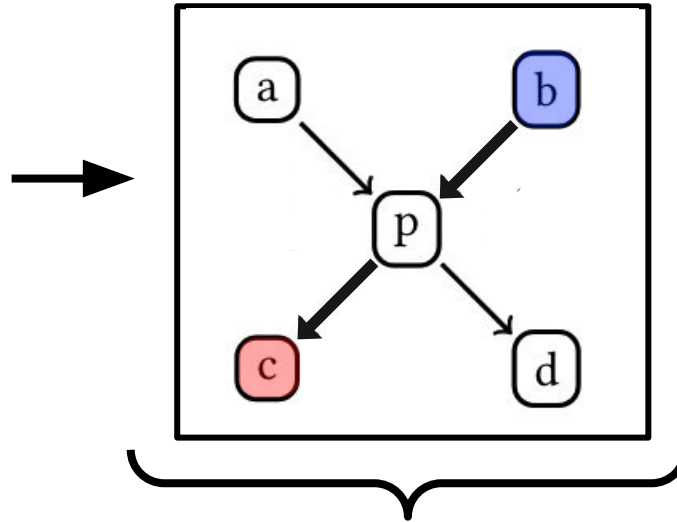
```
int a ← 2  
int b ← 3  
point p  
p.x ← a  
p.y ← b  
int c ← p.x  
int d ← p.y
```



What if there are composite data types?

Which variables affect the value of **c**?

```
int a ← 2  
int b ← 3  
point p  
p.x ← a  
p.y ← b  
int c ← p.x  
int d ← p.y
```

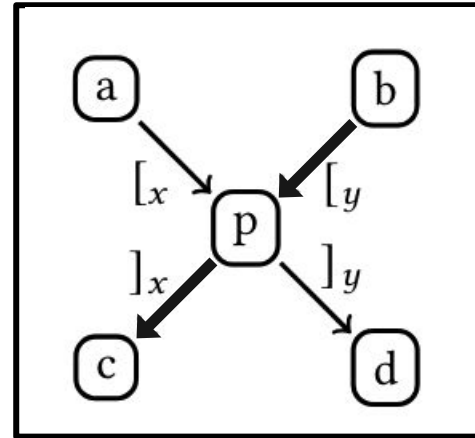


a reaches **c**... but so does **b**

Field-sensitivity

We use brackets to model fields

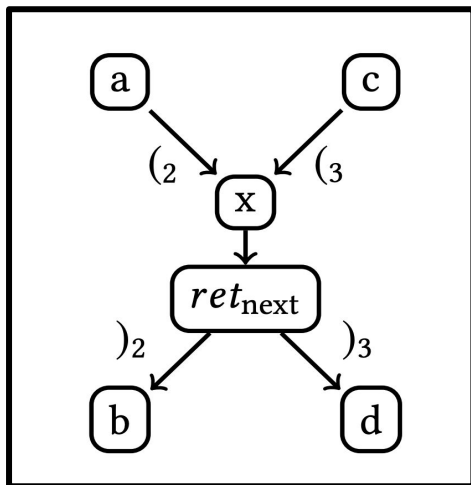
```
int  a ← 2  
int  b ← 3  
point p  
p.x ← a  
p.y ← b  
int  c ← p.x  
int  d ← p.y
```



Language reachability

Given a graph **G**, which pairs of vertices can reach other through a path in the language **L**

Widely used framework for static analysis

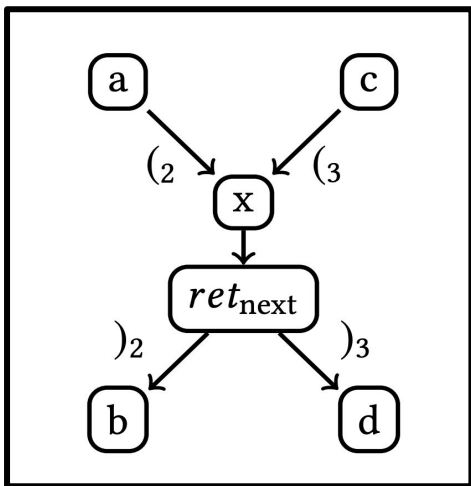


- **G** is the object graph
- **L** is the language of well-nested parentheses: **Dyck language**

Language reachability

Given a graph **G**, which pairs of vertices can reach other through a path in the language **L**.

Widely used framework for static analysis.



- **G** is the object graph
- **L** is the language of well-nested parentheses: **Dyck language**

this is a context-free grammar (**CFL**)

Language reachability

Can be defined through:

Dyck grammar

$$\mathcal{S}(\epsilon)$$

$$\mathcal{S}((ix)_i) \leftarrow \mathcal{S}(x)$$

$$\mathcal{S}(x_1x_2) \leftarrow \mathcal{S}(x_1), \mathcal{S}(x_2)$$

CFL reachability

Language reachability when the language is a CFL can be solved in **$O(n^3)$ time**

(good)

CFL reachability

Language reachability when the language is a CFL can be solved in **$O(n^3)$ time**

What about when it isn't?

CFL reachability

Language reachability when the language is a CFL can be solved in **$O(n^3)$ time**

What about when it isn't?

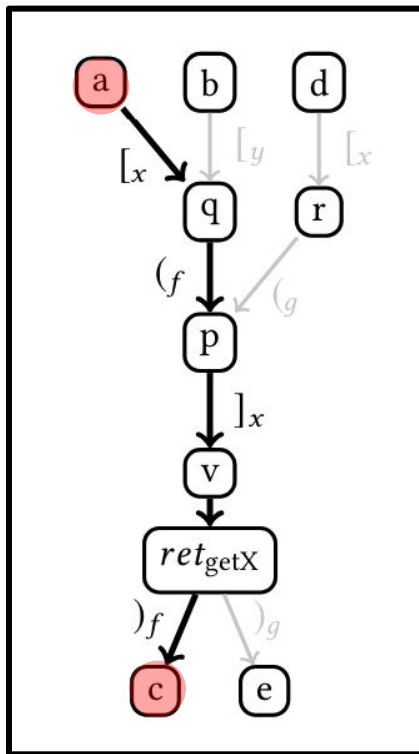
What about when we are dealing with multiple languages at once?

Context and field-sensitivity, simultaneously

```
fun getX(p)
  int v ← p.x
  return v

fun f()
  int a ← 2
  int b ← 3
  point q
  q.x ← a
  q.y ← b
  int c ← getX(q)

fun g()
  int d ← 1
  point r
  r.x ← d
  int e ← getX(r)
```

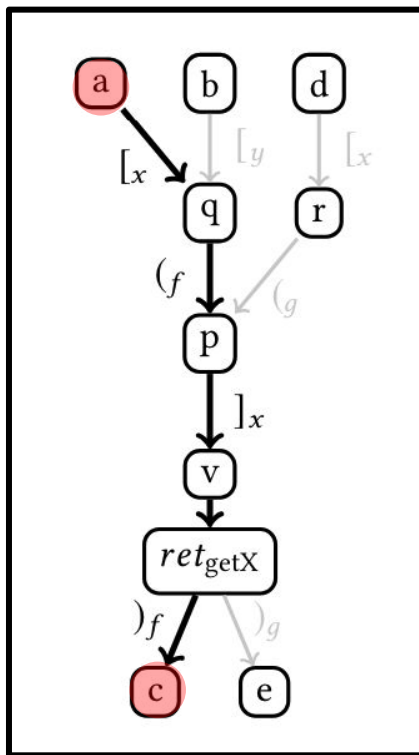


Context and field-sensitivity, simultaneously

```
fun getX(p)
  int v ← p.x
  return v

fun f()
  int a ← 2
  int b ← 3
  point q
  q.x ← a
  q.y ← b
  int c ← getX(q)

fun g()
  int d ← 1
  point r
  r.x ← d
  int e ← getX(r)
```



Parenthesis and brackets may be **interleaved**

$[_x ({}_f]_x)_f$

$[_x ({}_f]_x)_f$

$[_x ({}_f]_x)_f$

Context and field-sensitivity, simultaneously

This is called the Interleaved Dyck Language

- **IDL reachability is undecidable***

Therefore we must approximate it

- Many previous overapproximations (how good?)
- We present an **underapproximation**

(*) Undecidability of context-sensitive data-dependence analysis, **T. Reps**, 2000

MCFL reachability

Context-sensitive Languages

Context-free Languages

Dyck language English grammar*

Palindromes

$b^n a^{n+m} b^m$

Dutch grammar*

$s_1 s_2 s_2 s_1$

Word repeated twice

interleaved Dyck language

a^p , p is prime

*at least

Context-sensitive Languages

Multiple Context-free Languages

Context-free Languages

Dyck language

English grammar*

Palindromes

$b^n a^{n+m} b^m$

Dutch grammar*

$s_1 s_2 s_2 s_1$

Word repeated twice

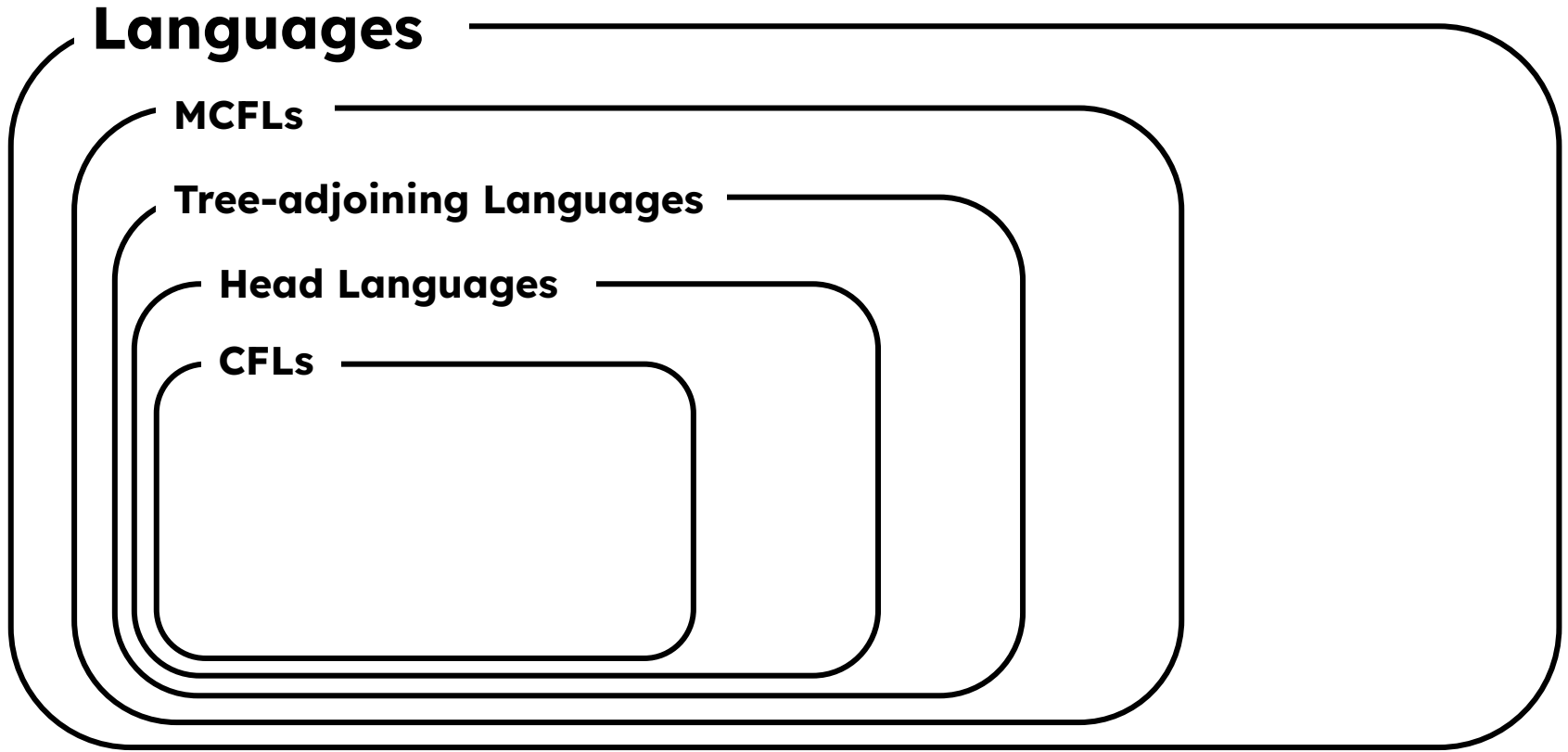
interleaved Dyck language

a^p , p is prime

→ We approximate IDL through an MCFL because MCFL-reachability is decidable

*at least

Languages



```
graph TD; Languages -- contains --> MCFLs; MCFLs -- contains --> TreeAdjoining[Tree-adjoining Languages]; TreeAdjoining -- contains --> HeadLanguages[Head Languages]; HeadLanguages -- contains --> CFLs
```

MCFLs

Tree-adjoining Languages

Head Languages

CFLs

CFLs

→ Rules and predicates

Language

$$\{0^n 1^n 1^m 0^m \mid n, m \geq 0\}$$



Grammar

$$A(\epsilon)$$

$$B(\epsilon)$$

$$A(0x1) \leftarrow A(x)$$

$$B(1x0) \leftarrow B(x)$$

$$S(x^1 x^2) \leftarrow A(x^1), B(x^2)$$

d-MCFL(r)

→ Rules and predicates

Language

$$\{w_1 w_2 \# w_2 w_1 \mid w_1, w_2 \in \{0, 1\}^*\}$$


Grammar

$$A(\epsilon, \epsilon)$$
$$A(x_1 0, x_2 0) \leftarrow A(x_1, x_2)$$
$$A(x_1 1, x_2 1) \leftarrow A(x_1, x_2)$$
$$S(x_1 y_1 \# y_2 x_2) \leftarrow A(x_1, x_2), A(y_1, y_2)$$

d-MCFL(r)

- Rules and predicates
- **Dimension d**

A produces **two** strings simultaneously, so **d=2**

Language

$$\{w_1 w_2 \# w_2 w_1 \mid w_1, w_2 \in \{0, 1\}^*\}$$


Grammar

$A(\epsilon, \epsilon)$

$$A(x_1 0, x_2 0) \leftarrow A(x_1, x_2)$$
$$A(x_1 1, x_2 1) \leftarrow A(x_1, x_2)$$
$$S(x_1 y_1 \# y_2 x_2) \leftarrow A(x_1, x_2), A(y_1, y_2)$$

d-MCFL(r)

- Rules and predicates
- **Dimension d**
- **Rank r**

A produces **two** strings simultaneously, so **d=2**

There are **two** predicates on the RHS, so **r=2**

Language

$$\{w_1 w_2 \# w_2 w_1 \mid w_1, w_2 \in \{0, 1\}^*\}$$



Grammar

$$A(\epsilon, \epsilon)$$

$$A(x_1 0, x_2 0) \leftarrow A(x_1, x_2)$$

$$A(x_1 1, x_2 1) \leftarrow A(x_1, x_2)$$

$$\mathcal{S}(x_1 y_1 \# y_2 x_2) \leftarrow A(x_1, x_2), A(y_1, y_2)$$

d-MCFL(r)

- Rules and predicates
- **Dimension d**
- **Rank r**
- For $d=1$, it is a CFL

A produces **two** strings simultaneously, so **d=2**

There are **two** predicates on the RHS, so **r=2**

Language

$$\{w_1 w_2 \# w_2 w_1 \mid w_1, w_2 \in \{0, 1\}^*\}$$



Grammar

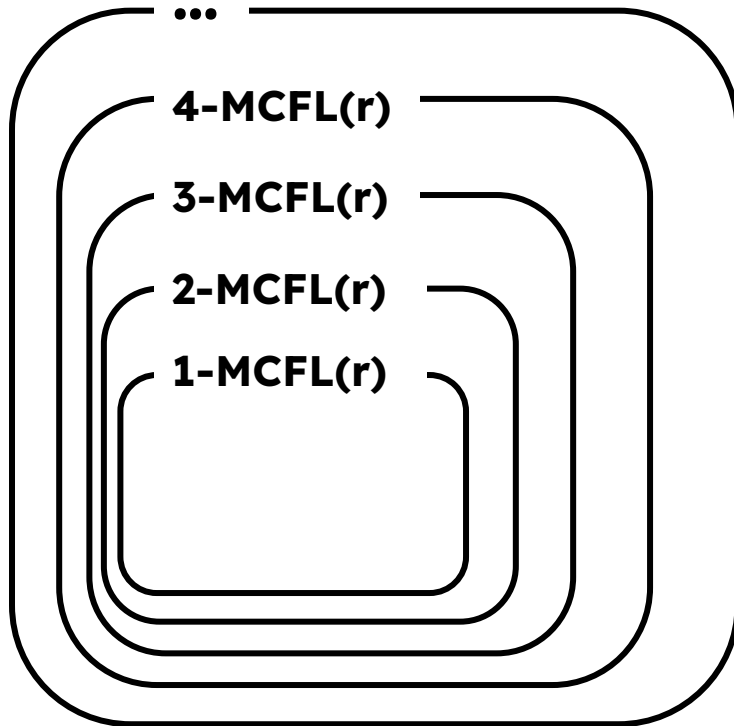
$$A(\epsilon, \epsilon)$$

$$A(x_1 0, x_2 0) \leftarrow A(x_1, x_2)$$

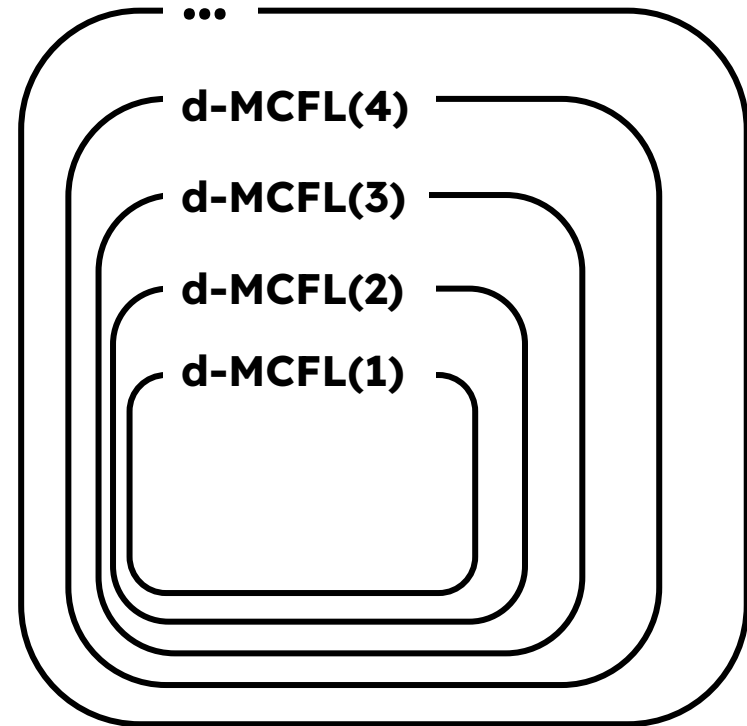
$$A(x_1 1, x_2 1) \leftarrow A(x_1, x_2)$$

$$\mathcal{S}(x_1 y_1 \# y_2 x_2) \leftarrow A(x_1, x_2), A(y_1, y_2)$$

For $r \geq 1$



For $d \geq 2$



Approximating IDL

([)], (([)]), ()[), [([([)])])]

Strings in IDL (labels omitted)

Can we underapproximate this using a CFL?

Approximating IDL as a CFL

Concatenate a parenthesis string and a bracket string

S(**(****(****(****(****)****)****)****)****[****[****]****]****[****]****)**

Can't recognize strings where parenthesis and brackets interact

(**[****)****]**

How do we capture this "interleaving"?

Approximating IDL as MCFL

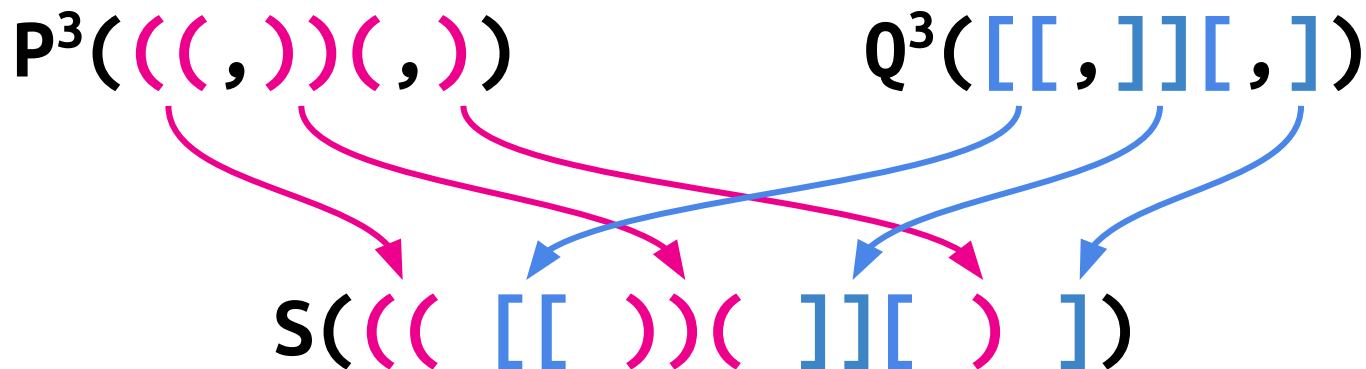
Idea: use dimensions as "breaking points" for interleaving

$P^3(((,)) (,))$

$Q^3([[,]][,])$

Approximating IDL as MCFL

$$S(x_1y_1x_2y_2x_3y_3) \leftarrow P^3(x_1, x_2, x_3), Q^3(y_1, y_2, y_3)$$



Tunable: the more dimensions, the more strings we can capture
Similar approach to bounded context-switchings

Approximating IDL as MCFL

- **BUT** we can add more rules, making the language a more complete underapproximation of IDL

$$P^3(x_1, yx_2, x_3) \leftarrow P^3(x_1, x_2, x_3), S(y)$$

- Even two dimensions can recognize **unbounded** context switchings

((([)][])([)]([[[[]]]))

Approximating IDL as MCFL

We add more derivation rules to make this grammar (much) more expressive

	Length of string						
	2	4	6	8	10	12	14
Strings in $\mathcal{L}(\mathcal{G}_1^+)$	2	8	40	224	1,344	8,448	54,912
Strings in $\mathcal{L}(\mathcal{G}_2^+)$	2	10	70	588	5,544	56,612	612,686
Strings in $\mathcal{L}(\mathcal{G}_3^+)$	2	10	70	588	5,544	56,628	613,470
Strings in $\mathcal{I}_{\alpha,\beta}$	2	10	70	588	5,544	56,628	613,470

MCFL reachability

MCFL reachability

Now that we have an MCFL approximation, how do we solve reachability?

MCFL reachability

We can generalize the CFL-reachability algorithm:

- Define a normal form for MCFGs
- Generalize the algorithm to work with multi-dimensional predicates
- **Runs in $O(n^{d(r+1)})$**
- Prove lower bounds for rank 1
 - ◆ $O(n^{2d-\epsilon})$ for d -MCFL(1) membership (under SETH)

Experiments

Taint analysis

Benchmark:

Scalable and precise taint analysis for Android, **Huang et al**, 2015

Graph on objects: we report the number of pairs (u,v) such that u may affect the value of v

We offer an **underapproximation** and compare it to the best available overapproximation*

(*) A Better Approximation for Interleaved Dyck Reachability, **G. Conrado, A. Pavlogiannis**, 2024

Taint analysis

Benchmark	Nodes	Edges	Overapp.
backflash	544	2,048	2,625
batterydoc	1,674	4,790	3,340
droidkongfu	734	1,983	3,401
fakebanker	434	1,103	251
fakedaum	1,144	2,603	2,282
faketaobao	222	450	59
jollyserv	488	998	164
loozfon	152	323	93
roidsec	553	2,026	13,052
uranai	568	1,246	143
zertsecurity	281	710	794
scipix	1,809	5,820	TO

Taint analysis

Benchmark	Nodes	Edges	Overapp.
backflash	544	2,048	2,625
batterydoc	1,674	4,790	3,340
droidkongfu	734	1,983	3,401
fakebanker	434	1,103	251
fakedaum	1,144	2,603	2,282
faketaobao	222	450	59
jollyserv	488	998	164
loozfon	152	323	93
roidsec	553	2,026	13,052
uranai	568	1,246	143
zertsecurity	281	710	794
scipix	1,809	5,820	TO

MCFL Underapprox.

1 dim	2 dim
2,625	2,625
2,804	2,839
2,906	3,361
249	251
1,132	2,276
57	59
155	164
76	93
12,284	13,052
143	143
779	794
1,699	2,115

Taint analysis

- Optimal for **8/11** benchmarks
- We confirm **97.9%** of the taints
- Uncertain pairs reduced to less than **2.1%**

MCFL
Underapprox.

1 dim	2 dim
2,625	2,625
2,804	2,839
2,905	3,361
249	251
1,132	2,276
57	59
155	164
75	93
12,284	13,052
143	143
779	794
1,699	2,115

Taint analysis

MCFL
Underapprox.

Benchmark	Nodes	Edges	Overapp.	Time	1 dim	2 dim	Time
backflash	544	2,048	2,625	2m 51.5s	2,625	2,625	1m 34.9s
batterydoc	1,674	4,790	3,340	30m 15.5s	2,804	2,839	38.1s
droidkongfu	734	1,983	3,401	2h 18m 30.4s	2,906	3,361	28m 48s
fakebanker	434	1,103	251	12.5s	249	251	3.9s
fakedaum	1,144	2,603	2,282	3m 5.2s	1,132	2,276	50m 47.2s
faketaobao	222	450	59	8.2s	57	59	0.8s
jollyserv	488	998	164	6.2s	155	164	1.5s
loozfon	152	323	93	4.2s	76	93	3.6s
roidsec	553	2,026	13,052	3m 14s	12,284	13,052	11m 6.5s
uranai	568	1,246	143	4.1s	143	143	1.6s
zertsecurity	281	710	794	9.6s	779	794	20.3s
scipix	1,809	5,820	TO	TO	1,699	2,115	7m 16.6s

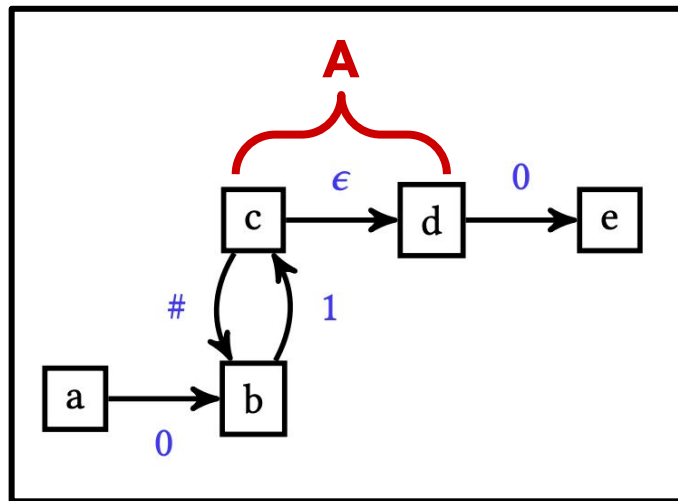
Conclusion

- Developed an algorithm for MCFL reachability
- Underapproximated Interleaved Dyck Reachability as a tunable MCFL
- Proved experimentally that this grammar is a good approximation, often producing optimal results

Showed that MCFLs can be a tool for static analysis, and provided a foundation for their use in the field

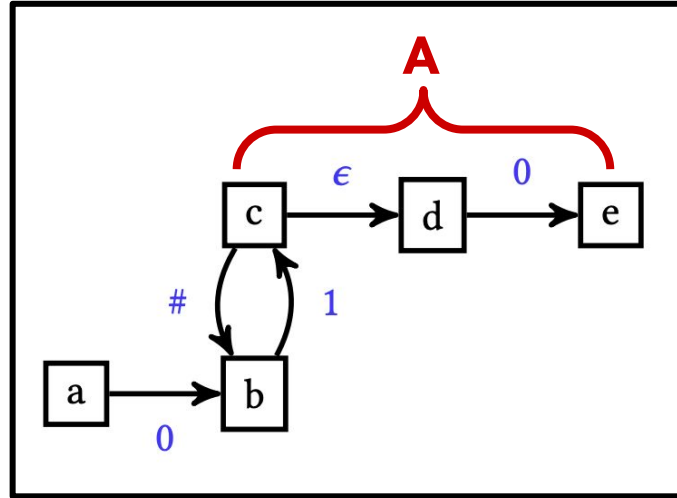
CFL reachability

$A(\epsilon)$

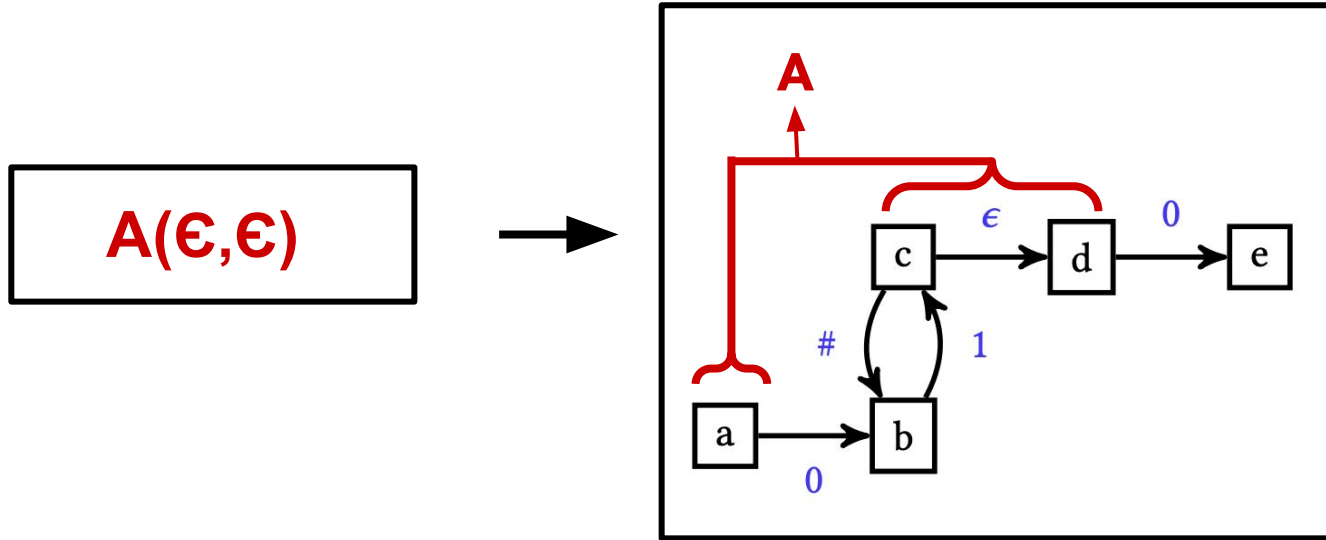


MCFL reachability

$A(x0) \leftarrow A(x)$

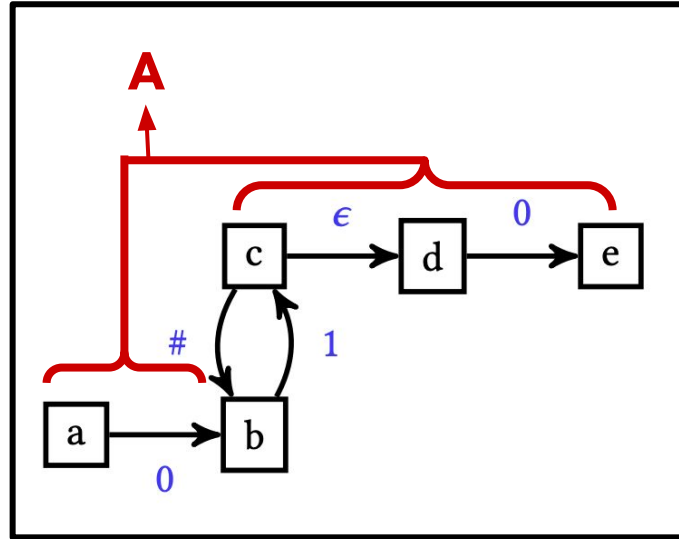


MCFL reachability



MCFL reachability

$A(x0, x0) \leftarrow A(x, x)$



Comparison with MPDS approximations

- Stronger than bounded context-switchings for same number of contexts
- Incomparable to bounded phases
 - ◆ MCFL of dimension $\mathbf{d}$ recognizes strings not recognized by $\mathbf{k}$ phases for any $\mathbf{k}$
 - ◆ $\mathbf{k}$ phases recognizes strings not recognized by an MCFL of dimension $\mathbf{d}$ for any $\mathbf{d}$

Comparison with MPDS approximations

- MCFL reachability has a single exponential dependency on the number of dimensions **d**
- Reachability for bounded phase multi-stack pushdown automata has a double exponential dependency on the number of phases **k**